```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>


class Student{
    private:
        std::pair<int, int> name;
        bool vaccinated;


public:
    //constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v){}


    // Getters
    auto get_name() { return name; }
    auto is_vaccinated() { return vaccinated; }


    // Two people are same if they have the same name
    bool operator ==(const Student& p) const {
        return this->name == p.name;
    }


    // The ordering of a set of people is defined by their name
    bool operator< (const Student& p) const {
        return this->name < p.name;
    }


    bool operator> (const Student& p) const {
        return this->name > p.name;
    }
};


auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());


    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);


    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}


bool needs_vaccination(Student P, std::vector<Student>& people) {
```

```
    auto first = people.begin();
    auto last = people.end();


    while (first != last){
        auto range_length = std::distance(first, last);
        auto mid_element_index = range_length / 2;
        auto mid_element = *(first + mid_element_index);


        // Return true if the Student is found in the sequence and
        // he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;


        else if (mid_element == P && mid_element.is_vaccinated() == true)
            return false;


        else if (mid_element > P)
            last = first + mid_element_index;


        else if (mid_element < P)
            first = first + mid_element_index + 1;
    }


    // Student not found in the sequence and therefore should be vaccinated
    return true;
}


void search_test(int size, Student p) {
    std::vector<Student> people;


    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());


    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point
    begin = std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);


    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    std::cout << "Time taken to search = " <<
        std::chrono::duration_cast<std::chrono::microseconds>
        (end - begin).count() << " microseconds" << std::endl;


    if (search_result){
```

```
        std::cout << "Student (" << p.get_name().first << " " <<
p.get_name().second << ") " << "needs vaccination." << std::endl;
    }
    else {
        std::cout << "Student (" << p.get_name().first << " " <<
p.get_name().second << ") " << "does not need vaccination." << std::endl;
    }
}

int main() {

    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```